

SpringBoot企业级后端开发技术手册

Java + SpringBoot + MySQL + Redis 集成实践

文档版本：V1.0

编制日期：2026年09月05日

适用范围：企业级Java后端开发、知识库系统测试

文档密级：内部公开

修订历史

版本	修订日期	修订内容	修订人
V1.0	2026-09-05	初始版本，完成架构、配置、MySQL、Redis、代码示例全章节编写	技术部

目 录

1 概述

- 1.1 编写背景
- 1.2 文档目标
- 1.3 适用读者

2 技术栈与整体架构

- 2.1 核心技术栈选型
- 2.2 系统分层架构设计
- 2.3 各层职责说明

3 请求完整流转流程

- 3.1 带缓存的请求处理流程
- 3.2 缓存一致性策略

4 SpringBoot工程基础配置

- 4.1 Maven核心依赖配置
- 4.2 application.yml全局配置
- 4.3 项目启动类与包结构

5 MySQL数据库实践

- 5.1 表结构设计规范
- 5.2 索引优化最佳实践
- 5.3 慢查询排查与优化

6 Redis缓存集成实践

- 6.1 RedisTemplate配置与使用
- 6.2 缓存穿透、击穿、雪崩解决方案

6.3 缓存更新策略对比

7 分布式锁实现

7.1 Redisson集成配置

7.2 分布式锁业务代码示例

8 业务代码完整示例

8.1 实体类与DTO定义

8.2 Controller控制层

8.3 Service业务层（含缓存逻辑）

8.4 Mapper数据访问层

9 故障排查与运维

9.1 Redis常见故障排查

9.2 MySQL常见故障排查

9.3 线上问题排查流程

10 性能压测与优化

10.1 核心性能指标

10.2 系统优化方案

参考文献

1 概述

1.1 编写背景

随着互联网业务规模的不断增长，企业级后端系统对高并发、高可用、高性能的要求日益提高。SpringBoot作为当前Java生态中最主流的后端开发框架，配合MySQL关系型数据库与Redis缓存中间件，已成为构建企业级应用的标准技术组合。然而在实际开发过程中，开发人员常常面临配置混乱、缓存使用不当、数据库性能瓶颈、缓存与数据库一致性问题。

本文档旨在系统性地梳理SpringBoot + MySQL + Redis技术栈的最佳实践，从架构设计、基础配置、数据库优化、缓存集成、分布式锁到故障排查，提供可直接落地的代码示例与解决方案，同时作为知识库系统的标准测试文档，验证文档解析、分块、检索与问答效果。

1.2 文档目标

- 提供一套完整的企业级SpringBoot后端项目架构方案；
- 规范MySQL数据库表设计、索引优化与慢查询排查流程；
- 详解Redis缓存的正确使用方式，解决缓存穿透、击穿、雪崩三大经典问题；
- 给出可直接复用的业务代码模板，包含Controller、Service、Mapper各层实现；
- 建立线上故障排查的标准化流程，缩短问题定位时间。

1.3 适用读者

本文档适用于以下人群：Java后端开发工程师、系统架构师、测试工程师、运维工程师，以及需要进行RAG知识库系统测试的技术人员。读者应具备基本的Java编程基础与Spring框架使用经验。

2 技术栈与整体架构

2.1 核心技术栈选型

本技术方案采用业界成熟稳定的开源技术组件，各组件版本均经过生产环境验证，具体选型如下表所示：

组件类别	技术选型	版本	核心用途
开发语言	Java	17 LTS	业务逻辑开发，支持虚拟线程等新特性
后端框架	SpringBoot	3.2.x	自动配置、快速集成、内嵌Tomcat
ORM框架	MyBatis-Plus	3.5.3	简化数据库CRUD操作，支持分页插件
关系型数据库	MySQL	8.0	业务数据持久化存储，支持事务
缓存中间件	Redis	7.0	热点数据缓存、分布式锁、会话共享
分布式锁客户端	Redisson	3.23.x	提供可重入锁、公平锁、读写锁等实现
反向代理	Nginx	1.24.x	负载均衡、HTTPS终止、静态资源处理
连接池	HikariCP	5.x	高性能数据库连接池，SpringBoot默认

2.2 系统分层架构设计

系统采用经典的五层分层架构，自上而下分别为接入层、控制层、业务层、数据访问层与存储层，各层职责清晰，通过接口进行解耦，便于独立开发、测试与维护。整体架构如图1所示。



图1 SpringBoot企业级后端系统分层架构图

2.3 各层职责说明

2.3.1 接入层

接入层由Nginx承担，主要负责请求的反向代理、负载均衡、HTTPS证书终止、静态资源缓存以及跨域处理。通过Nginx可以将请求均匀分发到多个SpringBoot应用节点，实现系统的水平扩展。同时Nginx还可以配置限流规则，防止恶意流量冲击后端服务。

2.3.2 控制层 (Controller)

控制层是系统对外的HTTP接口入口，主要负责接收用户请求、进行参数校验、调用业务层处理逻辑，并将处理结果统一封装为标准响应格式返回给前端。控制层不应包含复杂的业务逻辑，只做请求转发与响应封装。

2.3.3 业务层 (Service)

业务层是系统的核心，负责封装所有业务逻辑，包括事务控制、缓存处理、业务规则校验、外部服务调用等。业务层通过接口与实现类分离的方式进行开发，便于后续扩展与单元测试。对于需要事务的方法，使用 `@Transactional` 注解进行声明式事务管理。

2.3.4 数据访问层（DAO/Mapper）

数据访问层负责与数据库进行交互，执行SQL语句并将结果映射为Java对象。本方案采用MyBatis-Plus作为ORM框架，对于简单的CRUD操作可以直接继承BaseMapper接口无需编写SQL，对于复杂查询则在XML文件中编写自定义SQL。

2.3.5 存储层

存储层包含MySQL与Redis两个组件：MySQL作为主存储，负责所有业务数据的持久化，保证数据的一致性与可靠性；Redis作为缓存存储，负责存放高频访问的热点数据、分布式锁、用户会话等，以减轻数据库压力并提升系统响应速度。

3 请求完整流转流程

3.1 带缓存的请求处理流程

对于读多写少的业务场景，引入Redis缓存可以显著提升系统性能。一个完整的带缓存请求处理流程如图2所示，请求从用户发起开始，经过Nginx转发、Controller接收、Service层缓存判断，最终返回结果。



图2 带缓存的用户请求完整流转流程图

流程详细步骤说明如下：

1. **用户发起请求**：客户端通过浏览器或APP向服务端发起HTTP请求，请求中携带必要的参数与身份凭证；
2. **Nginx反向代理**：Nginx接收到请求后，根据负载均衡策略（如轮询、加权轮询、IP哈希）将请求转发到某一个后端应用节点；
3. **Controller参数校验**：Controller层接收到请求后，首先通过JSR-303校验注解对参数进行合法性校验，校验不通过直接返回参数错误响应；

4. **Service查询缓存**：业务层首先根据业务ID构造Redis缓存Key，查询Redis中是否存在对应的数据；
5. **缓存命中判断**：如果缓存中存在数据，则直接将缓存数据反序列化为Java对象返回，无需访问数据库；如果缓存未命中，则继续下一步；
6. **查询MySQL数据库**：通过Mapper层执行SQL查询MySQL数据库，获取业务数据；
7. **回写Redis缓存**：将从数据库查询到的数据序列化后写入Redis，并设置合理的过期时间，以便后续请求可以直接命中缓存；
8. **返回响应结果**：将业务数据封装为标准响应对象，经过Controller返回给前端客户端。

3.2 缓存一致性策略

缓存与数据库的一致性缓存使用中的核心问题。本方案采用"更新数据库后删除缓存"的Cache-Aside模式，该模式是业界最常用的一致性方案，实现简单且能在绝大多数场景下保证最终一致性。具体策略如下：

- **读请求**：先读缓存，缓存命中直接返回；缓存未命中则读数据库，然后将数据写入缓存；
- **写请求**：先更新数据库，更新成功后再删除对应的缓存Key；
- **删除失败处理**：如果删除缓存失败，通过重试机制或订阅MySQL binlog的方式进行异步删除，保证最终一致性。

注意：不推荐使用"更新缓存"的方式，因为在并发场景下容易出现脏数据。删除缓存是更安全的做法，下次读取时会自动从数据库加载最新数据。

4 SpringBoot工程基础配置

4.1 Maven核心依赖配置

项目采用Maven进行依赖管理，在pom.xml中需要引入SpringBoot父依赖、Web启动器、Redis启动器、MySQL驱动、MyBatis-Plus以及Redisson等核心依赖。完整的依赖配置如下：

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.2.5</version>
    <relativePath/>
  </parent>

  <groupId>com.example</groupId>
  <artifactId>springboot-demo</artifactId>
  <version>1.0.0</version>

  <properties>
    <java.version>17</java.version>
    <mybatis-plus.version>3.5.3.1</mybatis-plus.version>
    <redisson.version>3.23.5</redisson.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-redis</artifactId>
    </dependency>
    <dependency>
      <groupId>com.mysql</groupId>
      <artifactId>mysql-connector-j</artifactId>
      <scope>runtime</scope>
    </dependency>
    <dependency>
      <groupId>com.baomidou</groupId>
      <artifactId>mybatis-plus-spring-boot3-starter</artifactId>
      <version>${mybatis-plus.version}</version>
    </dependency>
    <dependency>
      <groupId>org.redisson</groupId>
```

```
        <artifactId>redisson-spring-boot-starter</artifactId>
        <version>${redisson.version}</version>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>
    <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
        <optional>true</optional>
    </dependency>
</dependencies>
</project>
```

4.2 application.yml全局配置

SpringBoot项目的全局配置统一放在application.yml文件中，包含服务器端口、数据源配置、Redis配置、MyBatis-Plus配置、日志配置等。配置示例如下：

```
server:
  port: 8080
  servlet:
    context-path: /api
  tomcat:
    max-threads: 200
    accept-count: 100

spring:
  application:
    name: springboot-demo
  datasource:
    driver-class-name: com.mysql.cj.jdbc.Driver
    url: jdbc:mysql://127.0.0.1:3306/demo_db?
    useUnicode=true&characterEncoding=utf8&useSSL=false&serverTimezone=Asia/
    Shanghai
    username: root
    password: your_password
    hikari:
      maximum-pool-size: 20
      minimum-idle: 5
      connection-timeout: 30000
  redis:
    host: 127.0.0.1
    port: 6379
    password: your_redis_password
    database: 0
    timeout: 5000ms
    lettuce:
      pool:
        max-active: 20
        max-idle: 10
        min-idle: 2

mybatis-plus:
  mapper-locations: classpath:mapper/*.xml
  configuration:
    map-underscore-to-camel-case: true
  global-config:
    db-config:
      id-type: auto
      logic-delete-field: deleted
      logic-delete-value: 1
      logic-not-delete-value: 0
```

```
logging:
  level:
    com.example.demo: debug
    org.springframework: info
```

4.3 项目启动类与包结构

项目启动类需要放在根包路径下，并添加 `@SpringBootApplication` 注解，如果需要扫描Mapper接口还需要添加 `@MapperScan` 注解。启动类代码如下：

```
package com.example.demo;

import org.mybatis.spring.annotation.MapperScan;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
@MapperScan("com.example.demo.mapper")
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
        System.out.println("应用启动成功: http://localhost:8080/api");
    }
}
```

标准的项目包结构如下所示，各包职责清晰，便于代码维护：

```
com.example.demo
├─ DemoApplication.java      # 启动类
├─ config                   # 配置类
├─ controller               # 控制层
├─ service                  # 业务层接口
│   └─ impl                 # 业务层实现
├─ mapper                   # 数据访问层
├─ entity                   # 数据库实体类
├─ dto                      # 数据传输对象
├─ vo                       # 视图返回对象
├─ common                   # 公共类
└─ utils                    # 工具类
```

5 MySQL数据库实践

5.1 表结构设计规范

良好的表结构设计是数据库性能的基础，在进行表设计时应遵循以下规范：

- **主键设计**：所有表必须有主键，推荐使用BIGINT自增主键，不使用业务字段作为主键；
- **必备字段**：所有业务表必须包含 `create_time`、`update_time`、`deleted` 三个字段；
- **字段类型**：字符串优先使用VARCHAR，金额使用DECIMAL，状态枚举使用TINYINT；
- **字段注释**：所有字段必须添加COMMENT注释，说明字段含义与取值范围；
- **避免大表**：单表数据量预计超过500万行时，应提前考虑分库分表方案。

以商品表为例，标准的建表SQL如下：

```
CREATE TABLE `t_goods` (  
  `id` BIGINT NOT NULL AUTO_INCREMENT COMMENT '主键ID',  
  `goods_name` VARCHAR(128) NOT NULL COMMENT '商品名称',  
  `goods_desc` VARCHAR(512) DEFAULT NULL COMMENT '商品描述',  
  `price` DECIMAL(10,2) NOT NULL DEFAULT '0.00' COMMENT '商品价格',  
  `stock` INT NOT NULL DEFAULT '0' COMMENT '库存数量',  
  `category_id` BIGINT NOT NULL COMMENT '分类ID',  
  `status` TINYINT NOT NULL DEFAULT '1' COMMENT '状态: 1-上架, 0-下架',  
  `create_time` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',  
  `update_time` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE  
CURRENT_TIMESTAMP COMMENT '更新时间',  
  `deleted` TINYINT NOT NULL DEFAULT '0' COMMENT '逻辑删除: 0-未删除, 1-已删除',  
  PRIMARY KEY (`id`),  
  KEY `idx_category_id` (`category_id`),  
  KEY `idx_status` (`status`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='商品表';
```

5.2 索引优化最佳实践

索引是提升数据库查询性能的最有效手段，但索引并非越多越好，不合理的索引会降低写入性能并占用存储空间。索引优化的常见场景与方案如下表所示：

问题场景	优化方案	原理说明
查询条件未命中索引	建立联合索引，遵循最左前缀原则	联合索引按字段顺序构建B+树，必须从最左字段开始匹配
LIKE '%xxx'前缀模糊查询	改用ElasticSearch全文检索引擎	前缀模糊查询无法利用B+树索引，会导致全表扫描
对索引列使用函数/运算	将运算移到条件右侧，或建立函数索引	对索引列使用函数会导致索引失效
隐式类型转换	保证查询参数类型与字段类型一致	字符串字段用数字查询会触发隐式转换，导致索引失效
大量更新的字段	不要在频繁更新的字段上建立索引	每次更新都会同时维护索引，严重影响写入性能
低区分度字段	避免在性别、状态等字段单独建索引	区分度低的字段过滤效果差，优化器可能选择全表扫描
ORDER BY/GROUP BY 字段	为排序和分组字段建立索引	索引本身有序，可以避免额外的filesort排序操作

5.3 慢查询排查与优化

当系统出现接口响应慢的问题时，首先应排查是否存在慢SQL。MySQL慢查询排查流程如下：

1. **开启慢查询日志**：设置 `slow_query_log=ON`，`long_query_time=0.2`（超过200ms的SQL会被记录）；
2. **分析慢查询日志**：使用 `mysqldumpslow` 工具汇总分析，找出执行次数最多、耗时最长的SQL；
3. **查看执行计划**：对慢SQL使用 `EXPLAIN` 命令查看执行计划，重点关注type、key、rows、Extra四个字段；
4. **针对性优化**：根据执行计划结果，通过添加索引、改写SQL、调整表结构等方式进行优化；
5. **验证优化效果**：优化后再次执行SQL，查看执行时间与执行计划，确认优化生效。

EXPLAIN执行计划中常见的type类型性能从好到差依次为：`system > const > eq_ref > ref > range > index > ALL`。如果出现type为ALL，说明是全表扫描，必须进行优化。

6 Redis缓存集成实践

6.1 RedisTemplate配置与使用

SpringBoot提供了RedisTemplate来简化Redis操作，但默认的JdkSerializationRedisSerializer序列化方式会产生乱码且体积较大。推荐自定义RedisTemplate配置，使用StringRedisSerializer序列化Key，使用Jackson2JsonRedisSerializer序列化Value。配置类代码如下：

```
@Configuration
public class RedisConfig {
    @Bean
    public RedisTemplate<String, Object> redisTemplate(RedisConnectionFactory
factory) {
        RedisTemplate<String, Object> template = new RedisTemplate<>();
        template.setConnectionFactory(factory);
        Jackson2JsonRedisSerializer<Object> jacksonSerializer =
            new Jackson2JsonRedisSerializer<>(Object.class);
        ObjectMapper objectMapper = new ObjectMapper();
        objectMapper.setVisibility(PropertyAccessor.ALL,
JsonAutoDetect.Visibility.ANY);

        objectMapper.activateDefaultTyping(LaissezFaireSubTypeValidator.instance,
            ObjectMapper.DefaultTyping.NON_FINAL);
        jacksonSerializer.setObjectMapper(objectMapper);
        StringRedisSerializer stringSerializer = new StringRedisSerializer();
        template.setKeySerializer(stringSerializer);
        template.setHashKeySerializer(stringSerializer);
        template.setValueSerializer(jacksonSerializer);
        template.setHashValueSerializer(jacksonSerializer);
        template.afterPropertiesSet();
        return template;
    }
}
```

配置完成后，即可在Service层注入RedisTemplate进行缓存操作。基础的缓存读写工具类示例如下：

```
@Component
public class RedisUtils {
    @Autowired
    private RedisTemplate<String, Object> redisTemplate;

    public void set(String key, Object value, long timeout, TimeUnit unit) {
        redisTemplate.opsForValue().set(key, value, timeout, unit);
    }
    public Object get(String key) {
        return redisTemplate.opsForValue().get(key);
    }
    public Boolean delete(String key) {
        return redisTemplate.delete(key);
    }
    public Boolean expire(String key, long timeout, TimeUnit unit) {
        return redisTemplate.expire(key, timeout, unit);
    }
    public Boolean hasKey(String key) {
        return redisTemplate.hasKey(key);
    }
}
```

6.2 缓存穿透、击穿、雪崩解决方案

缓存使用不当会引发三大经典问题：缓存穿透、缓存击穿、缓存雪崩，这三类问题都可能导致数据库被突发流量打垮，严重时造成系统雪崩。三类问题的处理架构如图3所示。

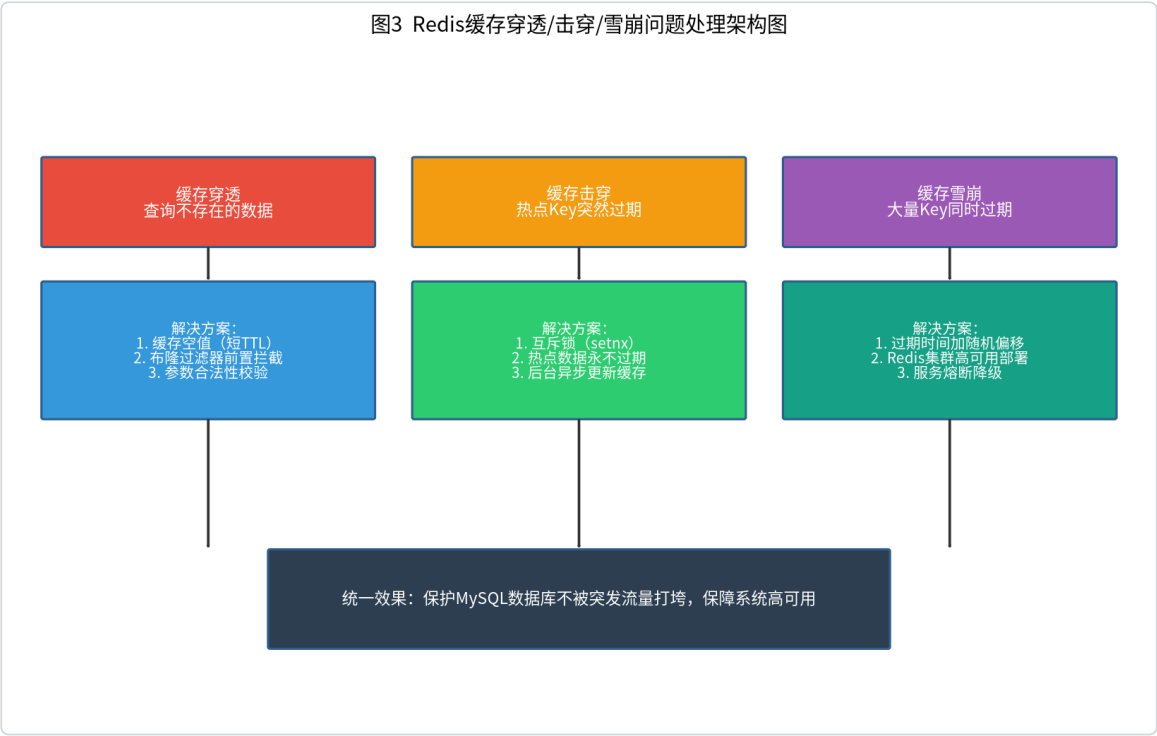


图3 Redis缓存穿透/击穿/雪崩问题处理架构图

三类问题的定义、产生原因与解决方案如下表所示：

问题类型	问题定义	产生原因	解决方案
缓存穿透	查询数据库中不存在的数据，缓存永远无法命中	恶意攻击、非法参数、业务数据不存在	1. 缓存空值并设置短TTL；2. 布隆过滤器前置拦截；3. 参数合法性校验
缓存击穿	某个热点Key突然过期，大量并发请求同时查库	热点数据缓存过期瞬间，高并发访问	1. 互斥锁只允许一个请求查库；2. 热点数据永不过期；3. 后台异步更新
缓存雪崩	大量Key同时过期，或Redis宕机，数据库压力骤增	过期时间相同、Redis单点故障	1. 过期时间加随机偏移；2. Redis集群高可用；3. 服务熔断降级

以缓存击穿为例，使用互斥锁解决的核心代码逻辑如下：

```
public GoodsV0 queryGoodsById(Long goodsId) {
    String key = "goods:info:" + goodsId;
    Object cacheValue = redisUtils.get(key);
    if (cacheValue != null) return (GoodsV0) cacheValue;
    String lockKey = "lock:goods:" + goodsId;
    try {
        boolean locked = redisTemplate.opsForValue()
            .setIfAbsent(lockKey, "1", 10, TimeUnit.SECONDS);
        if (locked) {
            GoodsV0 goodsV0 = goodsMapper.selectById(goodsId);
            if (goodsV0 != null) {
                redisUtils.set(key, goodsV0, 1, TimeUnit.HOURS);
            } else {
                redisUtils.set(key, null, 5, TimeUnit.MINUTES);
            }
            return goodsV0;
        } else {
            Thread.sleep(100);
            return queryGoodsById(goodsId);
        }
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
        return null;
    } finally {
        redisUtils.delete(lockKey);
    }
}
```

6.3 缓存更新策略对比

策略	读操作	写操作	一致性	适用场景
Cache-Aside	先读缓存，未命中读库并回写	先更新数据库，再删除缓存	最终一致	最通用，绝大多数业务场景
Read-Through	由缓存层负责读库并回写	先更新数据库，再删除缓存	最终一致	读多写少，对业务代码透明
Write-Through	先读缓存，未命中读库	同时更新缓存和数据库	强一致	对一致性要求极高的场景
Write-Behind	先读缓存，未命中读库	只更新缓存，异步批量写库	弱一致	写多读少，允许短暂不一致

7 分布式锁实现

7.1 Redisson集成配置

在分布式系统中，JDK自带的synchronized和ReentrantLock只能保证单个JVM内的线程安全，无法满足多节点部署下的互斥需求。Redis是实现分布式锁最常用的方案，而Redisson是Redis官方推荐的Java客户端，提供了完善的分布式锁实现。

引入Redisson依赖后，只需添加简单的配置类即可使用。配置类代码如下：

```
@Configuration
public class RedissonConfig {
    @Value("${spring.redis.host}")
    private String host;
    @Value("${spring.redis.port}")
    private String port;
    @Value("${spring.redis.password}")
    private String password;

    @Bean
    public RedissonClient redissonClient() {
        Config config = new Config();
        config.useSingleServer()
            .setAddress("redis://" + host + ":" + port)
            .setPassword(password)
            .setDatabase(0)
            .setConnectionPoolSize(20)
            .setConnectionMinimumIdleSize(5);
        return Redisson.create(config);
    }
}
```

7.2 分布式锁业务代码示例

以商品扣减库存为例，使用Redisson可重入锁保证并发场景下库存不会超卖。业务代码如下：

```

@Service
public class GoodsServiceImpl implements GoodsService {
    @Autowired
    private GoodsMapper goodsMapper;
    @Autowired
    private RedissonClient redissonClient;

    @Override
    public boolean deductStock(Long goodsId, Integer quantity) {
        String lockKey = "lock:deduct:stock:" + goodsId;
        RLock lock = redissonClient.getLock(lockKey);
        try {
            boolean locked = lock.tryLock(3, 30, TimeUnit.SECONDS);
            if (!locked) throw new RuntimeException("系统繁忙, 请稍后重试");
            Goods goods = goodsMapper.selectById(goodsId);
            if (goods == null || goods.getStock() < quantity) return false;
            LambdaUpdateWrapper<Goods> wrapper = new LambdaUpdateWrapper<>();
            wrapper.eq(Goods::getId, goodsId)
                .ge(Goods::getStock, quantity)
                .setSql("stock = stock - " + quantity);
            return goodsMapper.update(null, wrapper) > 0;
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            return false;
        } finally {
            if (lock.isHeldByCurrentThread()) lock.unlock();
        }
    }
}

```

Redisson看门狗机制：如果未指定锁的持有时间，Redisson会默认加锁30秒，并每隔10秒检查一次，如果业务还未执行完则自动续期到30秒，有效避免了业务执行时间过长导致锁提前释放的问题。

8 业务代码完整示例

本章以商品管理模块为例，给出从实体类到Controller的完整代码示例，包含缓存逻辑的实现，可作为项目开发的模板参考。

8.1 实体类与DTO定义

商品实体类Goods对应数据库t_goods表，使用Lombok注解简化Getter/Setter代码：

```
@Data
@TableName("t_goods")
public class Goods {
    @TableId(type = IdType.AUTO)
    private Long id;
    private String goodsName;
    private String goodsDesc;
    private BigDecimal price;
    private Integer stock;
    private Long categoryId;
    private Integer status;
    @TableField(fill = FieldFill.INSERT)
    private LocalDateTime createTime;
    @TableField(fill = FieldFill.INSERT_UPDATE)
    private LocalDateTime updateTime;
    @TableLogic
    private Integer deleted;
}
```

8.2 Controller控制层

```
@RestController
@RequestMapping("/goods")
public class GoodsController {
    @Autowired
    private GoodsService goodsService;

    @GetMapping("/{id}")
    public Result<GoodsV0> getGoodsById(@PathVariable Long id) {
        return Result.success(goodsService.getGoodsById(id));
    }
    @PostMapping
    public Result<Boolean> addGoods(@RequestBody GoodsV0 goodsV0) {
        return Result.success(goodsService.addGoods(goodsV0));
    }
    @PutMapping
    public Result<Boolean> updateGoods(@RequestBody GoodsV0 goodsV0) {
        return Result.success(goodsService.updateGoods(goodsV0));
    }
    @DeleteMapping("/{id}")
    public Result<Boolean> deleteGoods(@PathVariable Long id) {
        return Result.success(goodsService.deleteGoods(id));
    }
}
```

8.3 Service业务层（含缓存逻辑）

```
@Service
public class GoodsServiceImpl implements GoodsService {
    private static final String GOODS_CACHE_PREFIX = "goods:info:";
    @Autowired
    private GoodsMapper goodsMapper;
    @Autowired
    private RedisUtils redisUtils;

    @Override
    public GoodsVO getGoodsById(Long id) {
        String cacheKey = GOODS_CACHE_PREFIX + id;
        Object cacheValue = redisUtils.get(cacheKey);
        if (cacheValue != null) return (GoodsVO) cacheValue;
        Goods goods = goodsMapper.selectById(id);
        if (goods == null) {
            redisUtils.set(cacheKey, null, 5, TimeUnit.MINUTES);
            return null;
        }
        GoodsVO goodsVO = new GoodsVO();
        BeanUtils.copyProperties(goods, goodsVO);
        redisUtils.set(cacheKey, goodsVO, 1, TimeUnit.HOURS);
        return goodsVO;
    }

    @Override
    public boolean updateGoods(GoodsVO goodsVO) {
        Goods goods = new Goods();
        BeanUtils.copyProperties(goodsVO, goods);
        int rows = goodsMapper.updateById(goods);
        if (rows > 0) {
            redisUtils.delete(GOODS_CACHE_PREFIX + goodsVO.getId());
        }
        return rows > 0;
    }
}
```

8.4 Mapper数据访问层

```
@Mapper
public interface GoodsMapper extends BaseMapper<Goods> {
    List<Goods> selectByCategoryId(@Param("categoryId") Long categoryId,
                                   @Param("offset") Integer offset,
                                   @Param("size") Integer size);
}
```

对应的Mapper XML文件：

```
<mapper namespace="com.example.demo.mapper.GoodsMapper">
    <select id="selectByCategoryId"
resultType="com.example.demo.entity.Goods">
        SELECT * FROM t_goods
        WHERE category_id = #{categoryId} AND deleted = 0 AND status = 1
        ORDER BY create_time DESC
        LIMIT #{offset}, #{size}
    </select>
</mapper>
```

9 故障排查与运维

9.1 Redis常见故障排查

Redis作为系统的核心缓存组件，一旦出现故障会严重影响系统性能。常见故障现象、原因与解决方案如下表：

故障现象	可能原因	排查与解决方案
Redis连接超时	网络不通、Redis宕机、连接池耗尽、防火墙限制	1. ping测试网络；2. 检查Redis进程；3. 查看连接池配置；4. 检查防火墙端口
Redis内存占用过高	缓存数据过多、大Key、过期Key未清理	1. info memory查看内存；2. bigkeys查找大Key；3. 调整maxmemory与淘汰策略
缓存与数据库不一致	删除缓存失败、并发读写、更新策略错误	1. 检查更新代码是否先更库再删缓存；2. 消息队列异步重试删除；3. 订阅binlog
Redis持久化阻塞	RDB/AOF重写fork耗时过长、内存不足	1. 查看slowlog；2. 调整持久化策略；3. 保证足够内存用于fork
缓存命中率低	Key设计不合理、过期时间过短、缓存数据量小	1. info stats查看命中率；2. 优化Key设计与过期时间；3. 扩大缓存覆盖

9.2 MySQL常见故障排查

故障现象	可能原因	排查与解决方案
数据库连接耗尽	连接池配置过小、长连接未释放、慢SQL占用连接	1. show processlist查看连接；2. 调大连接池；3. 排查慢SQL与连接泄漏
CPU占用率100%	大量慢SQL、全表扫描、锁等待严重	1. 慢查询日志定位SQL；2. 优化索引与SQL；3. kill异常连接
主从同步延迟	主库写入压力大、从库性能差、大事务、网络延迟	1. show slave status查看；2. 优化主库写入；3. 提升从库配置
死锁	事务加锁顺序不一致、索引不当锁范围过大	1. show engine innodb status查看；2. 统一加锁顺序；3. 优化索引
磁盘IO过高	大量随机IO、redo log刷盘频繁、查询无索引	1. iostat查看IO；2. 优化索引减少扫描；3. 调整innodb_io_capacity

9.3 线上问题排查流程

当线上系统出现异常时，应按照标准化流程进行排查，避免盲目操作导致问题扩大。标准排查流程如下：

1. **确认现象与影响范围**：明确是接口报错、响应慢还是服务不可用，影响的范围与业务模块；
2. **查看监控告警**：查看应用服务器CPU、内存、负载，数据库CPU、连接数、慢查询，Redis内存、命中率、连接数等监控指标；
3. **查看应用日志**：查看应用错误日志，定位异常堆栈与报错信息，确定是代码问题还是依赖组件问题；
4. **定位根因**：根据日志与监控信息，结合链路追踪工具，定位问题的根本原因；
5. **应急恢复**：优先恢复服务，可采取重启、扩容、降级、限流、回滚等应急措施；
6. **根因修复**：服务恢复后，针对根本原因进行代码修复或配置调整；
7. **复盘总结**：问题解决后进行复盘，编写故障报告，制定改进措施，避免同类问题再次发生。

10 性能压测与优化

10.1 核心性能指标

系统性能评估通常关注以下核心指标，在压测过程中需要重点监控：

指标名称	英文缩写	指标说明	参考标准
响应时间	RT	请求从发出到收到响应的时间	核心接口P99 < 500ms
每秒查询数	QPS	系统每秒能处理的请求数量	根据业务容量规划
每秒事务数	TPS	系统每秒能处理的事务数量	根据业务容量规划
错误率	Error Rate	请求失败数占总请求数的比例	< 0.1%
并发用户数	Concurrent Users	同时向系统发起请求的用户数量	根据业务峰值规划
CPU使用率	CPU Usage	服务器CPU占用百分比	< 70%
内存使用率	Memory Usage	服务器内存占用百分比	< 80%

10.2 系统优化方案

当系统性能不满足要求时，可以从以下几个层面进行优化：

10.2.1 应用层优化

- 合理使用缓存，将热点数据放入Redis，减少数据库访问；
- 避免在循环中调用数据库或远程接口，批量操作减少IO次数；
- 使用异步化处理非核心逻辑，如发送短信、记录日志等，提升主流程响应速度；
- 合理设置线程池参数，避免线程过多导致上下文切换开销；
- 使用连接池管理数据库与Redis连接，避免频繁创建销毁连接。

10.2.2 数据库层优化

- 为高频查询字段建立合适的索引，避免全表扫描；
- 优化SQL语句，避免SELECT *、避免大事务、避免深分页；
- 读写分离，读请求走从库，写请求走主库；
- 单表数据量过大时进行分库分表；
- 合理设置innodb_buffer_pool_size，提升缓存命中率。

10.2.3 架构层优化

- 服务无状态化，支持水平扩展，通过增加节点提升系统吞吐量；
- 接入层使用Nginx进行负载均衡与限流；
- 引入消息队列进行流量削峰，应对突发流量；
- 服务拆分，将单体应用拆分为微服务，独立部署与扩展；
- 多级缓存架构：本地缓存（Caffeine）+ 分布式缓存（Redis），进一步降低延迟。

参考文献

1. Spring Boot官方文档. <https://spring.io/projects/spring-boot>
2. Redis官方文档. <https://redis.io/documentation>
3. MySQL 8.0 Reference Manual. <https://dev.mysql.com/doc/refman/8.0/en/>
4. MyBatis-Plus官方文档. <https://baomidou.com/>
5. Redisson官方文档. <https://redisson.org/>
6. 《Redis设计与实现》，黄健宏著
7. 《高性能MySQL》，Baron Schwartz等著
8. 《Java并发编程实战》，Brian Goetz等著